

POS TECH

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 4 | AULA 04 -

CASOS DE USO E PADRÕES DE DESIGN SERVERLESS

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS 15

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VEM POR AÍ?

Criar aplicações serverless pode ser complexo, mas a adoção dessa arquitetura em escala cada vez maior tem ajudado a identificar padrões de implementação que podem auxiliar nesse processo, guiando a tomada de decisão. O desafio é criar uma solução personalizada que resolva os problemas apresentados por essa arquitetura.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

HANDS ON

Apesar dos desafios, existem muitas vantagens ao adotar uma arquitetura serverless. Nessa aula aprenderemos, através da arquitetura de solução, a balancear as vantagens e restrições que essa abordagem proporciona.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

SAIBA MAIS

A arquitetura serverless e a arquitetura distribuída

Se estamos pensando no desenvolvimento de aplicações serverless, precisamos pensar, também, em todo o aprendizado e experiências relacionadas à implementação de soluções em arquitetura **distribuída**.

O paradigma serverless, mesmo definindo uma solução com menos “criação” de infraestrutura, exige uma mudança fundamental na forma como pensamos e construímos sistemas. Não se trata apenas de “tirar um código do servidor e colocar na nuvem”, mas de uma reengenharia da lógica de negócio para componentes **coesos,** com **alta qualidade** e **desacoplados**.

Pensar em serverless é, em sua essência, abandonar a visão monolítica de uma aplicação e mudar para a ideia de um sistema como um **conjunto distribuído** de **“pedaços” de processamento**, muitas vezes chamados de **funções**. Cada um desses pedaços é responsável por uma única tarefa bem definida e limitada.

Pensando no momento da modelagem, devemos sempre lembrar que o desenho da arquitetura considera a fragmentação da lógica de negócio em unidades menores e cada um desses fragmentos (ou funções) é projetada para executar uma tarefa bem definida, em resposta a um evento específico.

Considere um sistema de e-commerce, por exemplo. Em uma arquitetura tradicional, o serviço de pedido poderia ser um módulo dentro de um grande aplicativo. Já em uma arquitetura serverless, a criação de um pedido seria uma função, a atualização do estoque seria outra, o envio de e-mail de confirmação, uma terceira, e o processamento de pagamento, uma quarta.

Essa granularidade extrema é a essência do serverless. Cada função é uma unidade de implantação, execução e escalonamento independente. Essa solução, se pensarmos de forma adequada na arquitetura, nos permite otimizar recursos e ter alta resiliência: se uma função falhar, ela não derruba todo o sistema. No entanto, exige uma modelagem cuidadosa para evitar o “monolito distribuído”, ou “microlito”, onde as funções são tão acopladas que a complexidade aumenta exponencialmente.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Portanto, ao aplicar as ideias do serverless, devemos ter três principais mudanças na mentalidade:

- **Fragmentação intencional:** funções são projetadas e construídas para serem isoladas, pequenas e focadas em uma responsabilidade única.
- **Composição via eventos:** ao invés de chamadas diretas de método ou API, muitas vezes uma função dispara eventos que outras funções escutam e processam.
- **Escalabilidade automática:** cada função escala de forma independente, de acordo com a carga associada ao seu evento de entrada.

A estrutura de gatilhos de operação: orquestrando a execução

Toda a operação desses “pedaços” de processamento, como se conectam e como são executados, se apoia nos gatilhos de operação. Uma função serverless não fica ativa o tempo todo esperando por trabalho; ela é disparada por um evento. Esses eventos são os “gatilhos” que iniciam a execução da função.

Exemplos comuns de gatilhos incluem:

• **HTTP/API Gateway:** uma requisição HTTP (REST API) que chega a um endpoint específico. É o gatilho mais comum para APIs serverless, muito do que se pensa sobre essa arquitetura atualmente está relacionado a ele.

• **Mensageria/filas:** uma mensagem que é publicada em uma fila (ex.: Azure Service Bus, Google Cloud Pub/Sub, AWS SQS). Ideal para processamento assíncrono.

• **Eventos de banco de dados:** uma alteração em um banco de dados (ex.: uma nova linha inserida no Cosmos DB, uma atualização no Firestore).

• **Eventos de armazenamento:** um novo arquivo carregado em um bucket de armazenamento (ex.: Google Cloud Storage, Azure Blob Storage, AWS S3).

• **Eventos agendados (Timers/Cron):** execução da função em intervalos de tempo definidos (ex.: cron jobs para relatórios diários).

• **Eventos de streaming:** processamento de dados em tempo real de fluxos (ex.: Kafka, Azure Event Hubs, Kinesis).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Os gatilhos **desacoplam** o produtor do evento do consumidor da função. A função não precisa saber quem a chamou ou de onde veio o evento, ela apenas reage a ele. Essa arquitetura, altamente baseada na ideia de resposta a eventos disparados e a emitir novos evento, é um pilar fundamental do serverless e promove resiliência e escalabilidade.

Um ponto de atenção: a orientação a eventos (e até mesmo a dependência de eventos) implica que **o fluxo de execução não é linear nem determinístico**, como em um monólito ou aplicação síncrona. Isso exige técnicas específicas de observabilidade e rastreamento, o que deve ser levado em consideração no projeto.

Padrões de tomada de decisão no design serverless

Ao projetar uma solução serverless, algumas decisões de design se tornam cruciais. Funções serverless possuem um **tempo máximo de execução (timeout)**. No Google Cloud Run, por exemplo, o padrão é 5 minutos, mas pode ser configurado. As funções devem ser rápidas e eficientes, concluindo sua tarefa bem antes do timeout e com o **mínimo de recursos necessário**. Se uma função exceder esse tempo, ela será encerrada, potencialmente levando a dados inconsistentes ou falhas no processo. Para resolver esse problema, podemos usar alguns padrões como o SAGA, que vamos abordar posteriormente.

Além disso, funções são efêmeras (stateless) e devem ser criadas tendo essa restrição como base. Isso significa que qualquer estado ou dado que precisa persistir deve ser armazenado externamente. Por isso, é necessário pensar na solução envolvendo **armazenamento de dados**, como bancos de dados, armazenamento de arquivo em nuvem, cache distribuído e, em casos de dados para execução dessas funções, usar serviços de gerenciamento de configuração e armazenamento de senhas (secrets).

É comum **compartilhar bancos de dados** (SQL ou NoSQL) entre diferentes funções. No entanto, temos alguns pontos de atenção que podem impactar diretamente o uso deste tipo de armazenamento em um ambiente com muita concorrência de acesso ao banco de dados. Se pensamos em segurança, cada função deve ter suas próprias credenciais, com **permissões mínimas necessárias**. Se pensamos em desempenho, o acesso aos recursos de dados pode ser um ponto de **latência** muito grande se a origem de dados se tornar instável devido ao uso. É

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

importante mensurar, com base em dados, qual a carga que o serviço de dados pode suportar e se é possível usar alguma solução que implemente CQRS para garantir o acesso aos dados de forma adequada, com pouco impacto para a aplicação.

Para arquivos grandes ou dados que não se encaixam bem em um banco de dados (imagens, arquivos de dados binários/proprietários), utilize serviços de **armazenamento de objetos** (Cloud Storage, Blob Storage, S3). Funções podem ler e escrever nesses buckets com uma velocidade muito boa e esses serviços não apresentam degradação de desempenho de acordo com o acesso que recebem.

Já para dados frequentemente acessados ou resultados de computações complexas, que podem impactar em tempo de espera ou uso intensivo de processamento (CPU), use um **cache distribuído** (Redis, Memcached) para reduzir a carga sobre o serviço de acesso de dados, possivelmente reduzindo a latência no acesso a dados mais distantes do ponto de processamento (como acesso a APIs externas) e melhorando a performance como um todo.

Por fim, funções devem ser **idempotentes**, ou seja, executar a mesma operação múltiplas vezes com o mesmo resultado. Para isso, elas precisam ser criadas para não reter estado. Isso é crucial em ambientes distribuídos onde mensagens duplicadas ou retentativas são comuns, como no uso de filas ou processos complexos de diversos passos. Por exemplo, se uma função de processamento de pagamento for disparada duas vezes para a mesma transação, ela deve garantir que a transação seja processada apenas uma vez.

Novo paradigma, novos desafios

Apesar das vantagens, a adoção do serverless apresenta algumas mudanças de paradigma em relação a questões que não eram consideradas anteriormente, bem como desafios que não são imediatamente óbvios, principalmente para quem está começando a desenvolver essas aplicações.

Embora o pagamento por uso seja atrativo e até mesmo usado como principal justificativa da implementação de aplicações serverless, o **custo total pode aumentar**, muitas vezes **consideravelmente** em cenários de alto volume e latência contínua. Funções que são chamadas com muita frequência, que demoram muito ou que precisam de uma quantidade de recursos muito grande para executar ou até que mantêm instâncias “quentes” (para evitar cold starts), podem se tornar mais caras que

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

um servidor virtual provisionado com capacidade ociosa, que são chamados (muitas vezes, injustamente) de “ralos de dinheiro”. O gerenciamento de custos se torna uma tarefa de otimização contínua, focada no consumo exato de recursos. Equipes de FinOps têm se especializado neste tipo de trabalho.

Além disso, monólitos são simples de entender, já que tudo está no mesmo lugar. São simples de alterar e de fazer deploy, porque são autocontidos. Aplicações serverless são **inerentemente distribuídas** e, em sua essência, desconectadas. Isso significa mais pontos de falha, mais serviços para monitorar e mais dependências para gerenciar. A **complexidade** não desaparece; ela se move da infraestrutura para a orquestração dos serviços e a depuração do fluxo de eventos, onde tão importante quanto a criação das funções é a organização de como elas se conectam e como rastrear sua execução, em caso de testes, correção de erro e análise de desempenho.

Portanto, temos um desafio extra: quando uma requisição passa por múltiplas funções e serviços de mensageria, faz muitas chamadas a diferentes serviços ou recebe chamadas de outros serviços/gatilhos, **rastrear** o caminho completo de uma transação ou diagnosticar a causa raiz de um erro (observabilidade) se torna um desafio significativo. Neste ponto, a solução passa, necessariamente, por implementar **logging estruturado e rastreamento distribuído (distributed tracing)**. Ferramentas como o Google Cloud Trace, Azure Application Insights ou AWS X-Ray são fundamentais para visualizar o fluxo das requisições através de várias funções e serviços.

Existem alguns padrões de projeto que se aplicam ao modo serverless de pensar as soluções, usados para resolver alguns dos problemas existentes nesse tipo de abordagem e que não são comuns em outros tipos arquitetura. Abordaremos esses padrões a seguir.

Padrão SAGA

Estamos acostumados a pensar em processamento de transação como uma linha do tempo reta, onde podemos voltar a um estado anterior de forma simples, fazendo um “rollback”. Mas, ao contrário das transações ACID (Atomicidade, Consistência, Isolamento, Durabilidade) em bancos de dados relacionais monolíticos, onde uma única transação garante que todas as operações sejam bem-sucedidas ou

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

revertidas em caso de falha, em um ambiente distribuído, essa garantia não existe, porque os serviços atuam de forma independente. É aqui que entra o **Padrão SAGA**.

O Padrão SAGA é uma maneira de **gerenciar transações distribuídas** em um ambiente onde cada serviço possui acesso a serviços de dados que não são sempre iguais a outros serviços no processo. Essencialmente, uma SAGA é uma **sequência de transações locais** (dentro de cada microsserviço/função), onde cada transação local efetua um processo que pode atualizar o estado de dados em um serviço externo (chamada de API, gravação em banco de dados, entre outros). Se uma transação local falhar em qualquer ponto da sequência, o SAGA executa um conjunto de **transações de compensação** para desfazer as alterações feitas pelas transações locais anteriores.

Existem duas abordagens principais para implementar SAGA: coreografia e orquestração.

Na **Coreografia (Choreography)**, cada serviço na SAGA publica eventos após a conclusão de sua transação local e outros serviços reagem a esses eventos, executando suas próprias transações locais, numa sequência predeterminada. Neste caso, não há um orquestrador central e os serviços são projetados para se comunicar e coordenar uns com os outros de forma descentralizada. Cada serviço sabe qual evento deve emitir para tratar cada caso e apenas repassa a informação para o próximo passo.

Na **Orquestração (Orchestration)**, existe um **orquestrador central (Saga Orchestrator)** que gerencia a sequência de transações locais. Neste modelo, o orquestrador envia comandos/eventos para cada serviço para executar sua transação local, orquestrando o processo. Os serviços respondem ao orquestrador com o resultado de sua operação e ele decide a próxima etapa com base nesses resultados ou aciona as transações de compensação em caso de falha.

CQRS

O **CQRS (Command Query Responsibility Segregation)** é um padrão arquitetural que propõe a separação das responsabilidades de leitura e escrita de dados em um sistema. É muito mais do que apenas separar bancos de dados em instâncias de leitura e escrita, é apontar procedimentos para um ou outro, de acordo com a necessidade.

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Ao invés de uma única camada de acesso a dados que executa tanto consultas (queries) quanto comandos (commands), o CQRS define dois modelos distintos: um modelo de **leitura** altamente otimizado para consultas (Query Model) e um modelo de **escrita** que gerencia comandos e mantém as regras de negócio (Command Model).

A separação em dois tipos de modelos permite que se defina, entre outros, qual o modo de funcionamento de cada componente em um sistema. Podemos, por exemplo, definir que todo o processamento de regras de negócio rode um modelo de Command Model, responsável por operar os dados diretamente, e toda a parte de geração de relatórios e consultas seja fornecido por um componente que funciona como Query Model, acessando origens de dados que podem estar pré-processadas, com restrições de exibição de dados ou até mesmo consumindo dados de origens diferentes daquelas que os componentes de escrita usam.

Neste caso, temos também o desafio da **sincronização** dos dados. Se temos apenas uma replicação simples de dados, não há um problema complexo, mas se usamos fontes de dados modeladas de formas distintas, precisamos definir como fazer a sincronização e qual a estratégia, que pode ser:

• **Síncrona**: após uma alteração, a atualização do modelo de leitura ocorre imediatamente.

• **Assíncrona**: utilização de eventos para atualizar o modelo de leitura. Isso gera uma consistência eventual, ou seja, os dados podem não estar consistentes com o estado atual refletido na origem dos dados.

Este padrão é especialmente útil em sistemas com alta complexidade de domínio (aplicações ricas em regras de negócio) e que possuem perfis de uso distintos para leitura e escrita, como e-commerces, marketplaces, plataformas financeiras, entre outros.

Event Sourcing

Um dos padrões muito citados quando falamos de serverless é o Event Sourcing, um padrão de persistência de dados de eventos que garante que todas as mudanças no estado de um aplicativo sejam armazenadas como uma sequência de eventos em uma estrutura de dados conhecida como **Event Store** (Repositório de Eventos). Em vez de salvar o “agora” (o estado atual), salvamos o “como chegamos aqui” (a série de eventos).

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

Um exemplo é a modelagem do processo de uma conta bancária. O saldo atual (o estado) é derivado de uma série de transações (eventos): depósitos, saques, transferências. Em um sistema tradicional, salvaríamos apenas o saldo final. Com Event Sourcing, salvaríamos cada depósito, saque e transferência como um evento individual. O saldo atual seria então calculado “reproduzindo” esses eventos desde o início.

Esse padrão apresenta muitas vantagens, como:

• **Auditoria completa:** você tem um registro completo e imutável de tudo o que aconteceu no sistema, permitindo que a história de cada execução seja auditada e, caso necessário, reprocessada.

• **Consistência e concorrência:** os eventos são gravados em uma ordem estrita e imutável e de acordo com a sequência que aconteceram, garantindo o isolamento de cada cadeia de execução de um processamento.

• **Separação de preocupações (em conjunto com o CQRS):** voltando ao que citamos no tópico anterior, usualmente o Event Sourcing é combinado com o padrão Command Query Responsibility Segregation (CQRS), onde as operações de escrita (comandos que geram eventos) são separadas das operações de leitura (que consultam visões otimizadas do estado).

Apesar de ser um pouco mais complexo de projetar e implementar, este padrão tem sido muito usado em diferentes projetos, dado que há muitas ferramentas de alta qualidade para servir como Event Store, como o Kafka, e bibliotecas especializadas neste tipo de organização de execução de processos.

Mas, como todas as soluções de tecnologia, o Event Sourcing não é uma “bala de prata”. Sua complexidade inicial, no projeto e na implementação, é maior do que a persistência tradicional, com bancos de dados, e muitas vezes o desafio no desenho dos projetos é trabalhar com os dois tipos de armazenamento de dados ao mesmo tempo. No entanto, para sistemas que exigem que todos os processos sejam auditados, resiliência a falhas, capacidade de entender ou até reverter a estados passados e suporte a padrões como CQRS, o Event Sourcing é uma arquitetura poderosa.

Considerações finais

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

A arquitetura serverless, que tem sido vendida como uma solução final para resolver a maioria dos problemas de construção de software em nuvem, é uma ferramenta poderosa para construir sistemas modernos (seja em tecnologias usadas ou em técnicas de arquitetura) escaláveis e resilientes.

No entanto, exige um profundo entendimento de seus princípios, um olhar crítico para os desafios (evidentes e ocultos) e a aplicação de padrões de design adequados. Um projeto serverless mal executado ou planejado pode incorrer em falhas de operação tão graves quanto qualquer outro software construído com qualquer técnica arquitetural. O conhecimento e a experiência são, neste ponto, fundamentais.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

O QUE VOCÊ VIU NESTA AULA?

A arquitetura serverless passa por um momento de adoção massiva. Esse tipo de adoção sempre resulta em uma quantidade grande de casos de sucesso, mas também de fracasso no resultado dos projetos. A diferença entre eles está diretamente relacionada ao cuidado dispensado no desenho da arquitetura da solução, assim como as decisões tomadas para resolver os problemas particulares da solução.

O uso de padrões já estabelecidos e o conhecimento prévio sobre cada uma das restrições e oportunidades desse tipo de arquitetura nos possibilita navegar o projeto de forma mais tranquila, garantindo o sucesso da solução.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

REFERÊNCIAS

AWS. **AWS Well-Architected**. Disponível em: https://aws.amazon.com/pt/architecture/well-architected. Acesso em: 07 jul. 2025.

GOOGLE Cloud. **Modelo de arquitetura do Google Cloud**. Disponível em: https://cloud.google.com/architecture/framework?hl=pt-br. Acesso em: 07 jul. 2025.

INDRASIRI, K; SUHOTHAYAN, S. **Design Patterns for Cloud Native Applications**. [S.l.]: O'Reilly, 2021.

MICROSOFT. **Azure Architecture Center**. Disponível em https://learn.microsoft.com/en-us/azure/architecture. Acesso em: 07 jul. 2025.

NEWMAN, S. **Building Microservices**: Designing Fine-Grained Systems. [S.l.]: O'Reilly, 2021.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com

PALAVRAS-CHAVE

Serverless. Casos de uso. Padrões de design.

FIAP

PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com



PDF exclusivo para Michelle Ayumi Kashima - rm371026
michelleayumi14@gmail.com